ЛАБОРАТОРНА РОБОТА №8

Робота з БД через ORM

Мета. Підключити ORM-бібліотеку, відобразити модель даних на таблиці, реалізувати доступ до даних через репозиторій і порівняти ORM із «сирим» API.

Теоретичні відомості

1.1. Принципи об'єктно-реляційного відображення (ORM)

У сучасних програмних системах для зберігання та обробки структурованої інформації широко використовуються реляційні бази даних. Водночас прикладні програми, розроблені мовою C++, зазвичай оперують об'єктами класів і структур, які описують сутності предметної області.

Це зумовлює необхідність організації взаємодії між двома різними моделями представлення інформації: об'єктною моделлю програми та реляційною моделлю бази даних.

Відмінності між об'єктною та реляційною моделями

В об'єктно-орієнтованому програмуванні інформація представлена об'єктами, які мають певний стан і поведінку. Стан об'єкта визначається значеннями його полів, а поведінка – методами відповідного класу.

У реляційній моделі інформація організована у вигляді таблиць. Кожна таблиця містить рядки (записи) та стовпці (атрибути), а зв'язки між таблицями встановлюються за допомогою ключів.

Наприклад, для представлення обладнання в C++ можна визначити структуру:

```
struct Equipment
{
    int id;
    std::string name;
    std::string category;
    std::string status;
};
```

Відповідна таблиця реляційної бази даних може мати таку структуру:

```
CREATE TABLE Equipment (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    category TEXT NOT NULL,
    status TEXT NOT NULL
);
```

Попри зовнішню подібність, ці представлення мають принципові відмінності.

Характеристика	**Об'єктна модель C++**	**Реляційна модель**
Основна одиниця даних	Об'єкт	Рядок таблиці
Структура	Клас або структура	Таблиця
Властивості	Поля об'єкта	Стовпці таблиці
Ідентифікація	Ідентичність об'єкта, адреса або прикладний ідентифікатор	Первинний ключ

Зв'язки	Вказівники, посилання, контейнери, ідентифікатори	Зовнішні ключі
Обробка даних	Методи, функції, алгоритми C++	SQL-запити
Час існування	Визначається життєвим циклом об'єкта	Дані зберігаються незалежно від виконання програми

Особливо важливою є відмінність між життєвим циклом об'єкта та збереженням запису в базі даних.

Наприклад, створення об'єкта:

```
Equipment sensor{
    0,
    "Sensor A1",
    "Sensor",
    "active"
};
```

не означає автоматичного створення відповідного запису в базі даних. Об'єкт існує в пам'яті програми, тоді як запис таблиці повинен бути створений окремою операцією збереження.

Аналогічно, зміна поля об'єкта не обов'язково призводить до оновлення відповідного запису.

Необхідність узгодження цих моделей відома як об'єктно-реляційна невідповідність (Object-Relational Impedance Mismatch).

Вона проявляється не лише у відмінностях структури даних, а й у способах представлення зв'язків, керування ідентичністю об'єктів, обробки відсутніх значень і забезпечення цілісності інформації.

Поняття та призначення ORM

Object-Relational Mapping (ORM) – це технологія, яка забезпечує відображення об'єктів програми на структури реляційної бази даних та автоматизує перетворення інформації між цими представленнями.

Основне призначення ORM полягає у зменшенні кількості службового коду, необхідного для виконання типових операцій із базою даних.

За безпосереднього використання SQLite C API програміст повинен самостійно:

- підготувати SQL-запит;
- передати значення параметрів;
- виконати запит;
- отримати результати;
- перетворити значення стовпців на відповідні типи C++;
- звільнити ресурси, пов'язані з виконанням запиту.

ORM автоматизує значну частину цих дій на основі заздалегідь визначених правил відображення.

Узагальнену схему взаємодії можна представити так:



Програма звертається до ORM через методи та вирази C++. ORM використовує опис відповідності між об'єктами та таблицями для формування необхідних SQL-операцій і перетворення результатів.

При цьому SQL не зникає з процесу обробки даних. Він залишається основним механізмом взаємодії з реляційною СКБД, але значна частина його формування приховується всередині бібліотеки.

Основні принципи об'єктно-реляційного відображення

В основі ORM лежить визначення відповідності між елементами об'єктної та реляційної моделей.

1. Відображення типів на таблиці (Type-to-Table Mapping).

Клас або структура C++ пов'язується з певною таблицею бази даних. Наприклад, структура Equipment відповідає таблиці Equipment. Один екземпляр структури може представляти один запис таблиці.

2. Відображення полів на стовпці (Member-to-Column Mapping).

Поля структури пов'язуються зі стовпцями таблиці.

Наприклад:

- Equipment::id – стовпець id;
- Equipment::name – стовпець name;
- Equipment::category – стовпець category;
- Equipment::status – стовпець status.

Під час такого відображення враховуються типи даних, правила перетворення значень і визначені обмеження.

3. Відображення ідентифікаторів (Identity Mapping).

Для однозначної ідентифікації записів у реляційній базі даних використовуються первинні ключі. У C++ відповідне значення може зберігатися у звичайному полі об'єкта. Наприклад, поле id структури Equipment може відповідати первинному ключу таблиці. Завдяки цьому ORM може визначити, який саме запис необхідно отримати, змінити або видалити. При цьому слід розрізняти ідентифікатор запису та ідентичність об'єкта C++. Два різні об'єкти в пам'яті можуть містити однаковий первинний ключ і представляти один запис бази даних.

4. Відображення зв'язків (Relationship Mapping).

У реляційних базах даних зв'язки між таблицями зазвичай визначаються через зовнішні ключі. Наприклад, таблиця Maintenance може містити записи про обслуговування обладнання. Кожен такий запис належить певному пристрою:

```
struct Maintenance
{
    int id;
    int equipmentId;
    std::string serviceDate;
    std::string description;
};
```

Поле equipmentId відповідає зовнішньому ключу equipment_id, який посилається на Equipment(id).

Таким чином реалізується зв'язок «один-до-багатьох»: один пристрій може мати багато записів обслуговування.

5. Перетворення результатів запитів на об'єкти (Object Materialization).

Після виконання SQL-запиту ORM отримує значення стовпців і створює відповідні об'єкти C++.

Наприклад, рядок таблиці:

id	name	category	status
1	Sensor A1	Sensor	active

може бути представлений об'єктом:

```
Equipment sensor{
    1,
    "Sensor A1",
    "Sensor",
    "active"
};
```

Цей процес називається матеріалізацією об'єктів.

Він дозволяє програмі працювати з результатами запитів через звичайні типи C++, не виконуючи ручного перетворення кожного стовпця.

Основні операції ORM

Більшість ORM-бібліотек надають засоби виконання базових операцій над даними, які відповідають концепції CRUD:

Операція	Призначення	SQL-еквівалент
Create	Створення нового запису	INSERT
Read	Отримання записів	SELECT
Update	Оновлення запису	UPDATE
Delete	Видалення запису	DELETE

Окрім CRUD, ORM може надавати засоби формування складніших запитів: фільтрації, сортування, обмеження кількості результатів, агрегації та об'єднання таблиць. Конкретний синтаксис залежить від бібліотеки.

Наприклад, у sqlite_orm отримання всіх записів певного типу може виконуватися так:

```
auto equipment = storage.get_all<Equipment>();
```

У результаті програма отримує колекцію об'єктів Equipment, з якою можна працювати стандартними засобами C++.

Водночас використання ORM не означає автоматичної синхронізації стану об'єктів із базою даних. У бібліотеці sqlite_orm зміна поля об'єкта не призводить до оновлення відповідного запису, доки не буде явно виконана операція збереження змін.

Використання ORM має низку переваг.

1. Зменшення кількості службового коду. Програмісту не потрібно вручну виконувати всі операції підготовки SQL-запитів, читання результатів і перетворення значень.

2. Типізований доступ до даних. Робота з полями структур і класів C++ дозволяє використовувати механізми перевірки типів компілятором.

3. Підвищення читабельності коду. Операції збереження та отримання об'єктів можуть бути представлені через зрозумілі виклики методів.

4. Спрощення супроводу. Правила відповідності між об'єктами та таблицями визначаються централізовано, що зменшує дублювання коду.

5. Інтеграція з можливостями C++. ORM-бібліотеки можуть використовувати шаблони, стандартні контейнери, std::optional, виключення та RAII.

Водночас ORM має певні обмеження.

1. По-перше, ORM не усуває необхідності розуміти реляційну модель і SQL. Некоректно сформований запит може бути неефективним незалежно від того, чи створено його вручну, чи через ORM.

2. По-друге, автоматизація приховує частину внутрішніх операцій. Розробник повинен розуміти, коли виконується звернення до бази даних, які записи завантажуються та скільки ресурсів потребує операція.

3. По-третє, можливості ORM залежать від конкретної бібліотеки. Деякі складні або специфічні для СКБД операції можуть вимагати використання SQL безпосередньо.

4. По-четверте, ORM не забезпечує автоматичного виконання бізнес-правил. Перевірка допустимості операцій, обробка помилок і забезпечення узгодженості стану застосунку залишаються відповідальністю розробника.

Отже, ORM слід розглядати як інструмент організації доступу до даних, який доповнює можливості реляційної СКБД, а не замінює їх.

1.2. Бібліотека sqlite_orm для C++

Для реалізації об'єктно-реляційного відображення в C++ можна використовувати різні бібліотеки, які відрізняються підтримуваними СКБД, способами визначення моделей даних і механізмами формування запитів. У цій лабораторній роботі використовується sqlite_orm – бібліотека, призначена для організації взаємодії C++-застосунків із реляційною базою даних SQLite.

sqlite_orm – це бібліотека об'єктно-реляційного відображення для C++, яка дозволяє пов'язувати структури та класи програми з таблицями SQLite і виконувати операції з даними через типізований програмний інтерфейс.

Бібліотека побудована на використанні шаблонів C++ і має переважно заголовкову реалізацію (header-only). Це означає, що основні механізми ORM реалізовані в заголовкових файлах, які підключаються під час компіляції програми. Водночас для безпосередньої роботи з базою даних використовується SQLite, тому проєкт повинен мати відповідну залежність.

Основні особливості sqlite_orm:

- Використання звичайних типів C++. Моделі даних можна представляти структурами або класами без обов'язкового успадкування від спеціальних базових класів ORM.
- Декларативне відображення. Відповідність між полями C++ і стовпцями таблиць визначається окремо від опису моделей.
- Типізоване формування запитів. Для виконання багатьох операцій використовуються вирази C++, а не текстові SQL-запити.
- Підтримка CRUD-операцій. Бібліотека надає методи додавання, отримання, оновлення та видалення записів.
- Робота зі зв'язками між таблицями. Передбачені засоби опису зовнішніх ключів і виконання запитів до пов'язаних таблиць.
- Підтримка транзакцій. Декілька операцій можуть виконуватися як єдина логічна одиниця роботи.

Бібліотека sqlite_orm не є самостійною СКБД. Вона не зберігає дані у власному форматі та не замінює механізми SQLite. Її завдання – забезпечити зручний рівень взаємодії між C++-програмою та SQLite.

Визначення моделі даних

Першим етапом використання ORM є створення моделі даних, яка описує сутності предметної області.

Розглянемо систему обліку обладнання. Кожен пристрій характеризується ідентифікатором, назвою, категорією та поточним станом.

```
struct Equipment
{
    int id = 0;
    std::string name;
    std::string category;
    std::string status;
};
```

Структура Equipment не містить залежностей від SQLite або sqlite_orm. Вона є звичайним типом C++, об'єкти якого можуть використовуватися незалежно від механізму зберігання даних.

Наприклад:

```
Equipment sensor{
    0,
    "Sensor A1",
    "Sensor",
    "active"
};
```

Створений об'єкт існує в оперативній пам'яті програми. Для його збереження в базі даних необхідно визначити відповідність між структурою Equipment та таблицею SQLite.

Опис відображення між структурою та таблицею

У sqlite_orm відповідність між об'єктною та реляційною моделями визначається за допомогою функцій make_storage(), make_table() та make_column().

Приклад опису відображення:

```
#include <sqlite_orm/sqlite_orm.h>
using namespace sqlite_orm;
auto storage = make_storage(
    "equipment.db",
    make_table(
        "Equipment",
        make_column(
            "id",
            &Equipment::id,
            primary_key().autoincrement()
        ),
        make_column(
            "name",
            &Equipment::name
        ),
        make_column(
```

```
            "category",
            &Equipment::category
        ),
        make_column(
            "status",
            &Equipment::status
        )
    )
);
```

Функція make_storage() створює об'єкт сховища, який містить інформацію про базу даних і правила відображення. Перший аргумент – "equipment.db" – визначає шлях до файлу SQLite.

Функція make_table() описує таблицю та її стовпці.

Функція make_column() установлює відповідність між назвою стовпця таблиці та полем структури C++. Наприклад, make_column("name", &Equipment::name) означає, що стовпцю name таблиці відповідає поле name структури Equipment.

Вираз &Equipment::name є вказівником на член класу (pointer to member). Він дозволяє бібліотеці визначити, яке саме поле об'єкта необхідно використовувати під час читання або збереження даних.

Для стовпця id додатково визначено primary_key().autoincrement(). Це вказує, що стовпець є первинним ключем із підтримкою автоматичного генерування ідентифікаторів.

Опис відображення сам по собі ще не означає створення таблиці в базі даних. Він визначає схему, з якою надалі працюватиме ORM.

Створення та синхронізація структури бази даних

Після визначення відображення необхідно забезпечити відповідність структури бази даних описаній моделі. Для цього використовується метод:

```
storage.sync_schema();
```

Метод sync_schema() порівнює описану схему з фактичною структурою бази даних та виконує підтримувані операції узгодження. Зокрема, якщо таблиця відсутня, вона може бути створена автоматично. У розглянутому прикладі буде створена таблиця Equipment із відповідними стовпцями та первинним ключем.

Слід враховувати, що sync_schema() не є універсальним механізмом міграції баз даних. Складні зміни структури, особливо в системах, які вже містять важливі дані, потребують окремого планування.

Виконання CRUD-операцій

Після створення сховища та синхронізації схеми можна виконувати операції над об'єктами.

Додавання запису (Create). Для додавання нового об'єкта використовується метод insert():

```
Equipment sensor{
    0,
    "Sensor A1",
    "Sensor",
    "active"
};
int id = storage.insert(sensor);
```

Бібліотека формує відповідну операцію INSERT і передає значення полів об'єкта до SQLite. У наведеній моделі ідентифікатор генерується базою даних. Метод insert() повертає ідентифікатор створеного запису.

Отримання запису (Read). Для отримання об'єкта за первинним ключем використовується метод get():

```
Equipment item = storage.get<Equipment>(id);
```

Бібліотека виконує відповідний запит до таблиці та перетворює отриманий рядок на об'єкт Equipment. Якщо запис із заданим ідентифікатором відсутній, get() генерує виключення. Для ситуацій, коли відсутність запису є допустимим результатом, доцільно використовувати get_optional():

```
auto result =
    storage.get_optional<Equipment>(id);

if (result)
{
    std::cout << result->name << '\n';
}
else
{
    std::cout << "Equipment not found\n";
}
```

Метод повертає std::optional<Equipment>, що дозволяє явно перевірити наявність результату.

Оновлення запису (Update). Для зміни запису необхідно модифікувати відповідний об'єкт і викликати update():

```
Equipment item = storage.get<Equipment>(id);
item.status = "repair";
storage.update(item);
```

Зміна поля status впливає лише на об'єкт у пам'яті. Виклик update() забезпечує збереження нового значення в базі даних.

Видалення запису (Delete). Для видалення запису за первинним ключем використовується метод remove():

```
storage.remove<Equipment>(id);
```

У результаті виконується відповідна операція DELETE.

Отже, основні операції з базою даних представлені через методи, які працюють безпосередньо з типами C++.

Отримання колекцій об'єктів і формування запитів

У практичних застосунках часто необхідно отримувати не окремі записи, а множини об'єктів, які відповідають певним умовам.

Для отримання всіх записів таблиці використовується метод get_all():

```
std::vector<Equipment> equipment =
    storage.get_all<Equipment>();
```

У результаті формується контейнер std::vector, елементи якого є об'єктами Equipment.

Для фільтрації даних використовується конструкція where():

```
auto active = storage.get_all<Equipment>(
    where(c(&Equipment::status) == "active")
);
```

Цей вираз відповідає SQL-запиту з умовою WHERE status = 'active'

Для сортування записів використовується order_by():

```
auto sorted = storage.get_all<Equipment>(
    order_by(&Equipment::name)
);
```

Для обмеження кількості результатів – limit():

```
auto firstFive = storage.get_all<Equipment>(
    limit(5)
);
```

Умови можна комбінувати:

```
auto result = storage.get_all<Equipment>(
    where(c(&Equipment::status) == "active"),
    order_by(&Equipment::name),
    limit(5)
);
```

У цьому випадку SQLite виконує фільтрацію, сортування та обмеження кількості записів, після чого ORM формує колекцію об'єктів C++.

Результат get_all() повністю матеріалізується в пам'яті програми. Тому під час роботи з великими наборами даних необхідно враховувати обсяг результатів.

Відображення зв'язків між таблицями

Реляційні бази даних дозволяють установлювати зв'язки між таблицями за допомогою зовнішніх ключів.

У нашому прикладі один пристрій може мати декілька записів обслуговування.

Для представлення цих записів визначимо структуру:

```
struct Maintenance
{
    int id = 0;
    int equipmentId = 0;
    std::string serviceDate;
    std::string description;
};
```

Поле equipmentId містить ідентифікатор обладнання, до якого належить запис. У реляційній базі даних відповідний стовпець equipment_id може бути визначений як зовнішній ключ до Equipment(id). У sqlite_orm зовнішні ключі описуються під час визначення таблиць за допомогою конструкції foreign_key().

Під час роботи зі SQLite необхідно враховувати, що перевірка зовнішніх ключів повинна бути ввімкнена для відповідного з'єднання з базою даних.

Для отримання інформації з декількох пов'язаних таблиць використовуються операції об'єднання, зокрема JOIN.

Бібліотека sqlite_orm надає засоби формування таких запитів через вирази C++. Детальну реалізацію зв'язку між таблицями та відповідного запиту буде розглянуто в практичному прикладі виконання лабораторної роботи.

Транзакції та обробка помилок

Окремі операції з базою даних можуть бути логічно пов'язаними.

Наприклад, під час реєстрації нового обладнання необхідно створити запис про пристрій і додати початковий запис до журналу його обслуговування. Якщо перша операція

завершиться успішно, а друга – помилкою, база даних може опинитися в неповному з погляду предметної області стані. Для запобігання таким ситуаціям використовуються транзакції. Транзакція об'єднує декілька операцій у єдину логічну одиницю роботи.

Якщо всі операції завершуються успішно, зміни фіксуються. Якщо виникає помилка, зміни транзакції можуть бути скасовані.

У sqlite_orm передбачені засоби виконання транзакцій, зокрема метод transaction(). Під час використання транзакцій необхідно враховувати можливість виникнення виключень, пов'язаних із порушенням обмежень цілісності, блокуванням бази даних або іншими помилками виконання запитів.

Особливості використання sqlite_orm у C++-проєкті

Для роботи з sqlite_orm необхідно забезпечити наявність бібліотеки та її залежностей у середовищі розробки. У лабораторній роботі для керування залежностями використовується vcpkg, а для організації складання проєкту – CMake. Такий підхід дозволяє визначити залежності проєкту в конфігураційних файлах і забезпечити відтворюваність середовища розробки.

Важливо, що використання ORM не звільняє програміста від необхідності правильно організовувати структуру застосунку. Зокрема, не рекомендується розміщувати всі операції доступу до бази даних безпосередньо в коді користувацького інтерфейсу або основної бізнес-логіки.

Для відокремлення таких операцій може використовуватися патерн Repository

1.3. Організація доступу до даних за допомогою Repository

Використання ORM дозволяє суттєво спростити виконання операцій із базою даних. Однак сама по собі ORM-бібліотека не визначає, яким чином повинен бути організований програмний код застосунку. У невеликих програмах операції отримання, додавання, оновлення та видалення записів можуть виконуватися безпосередньо у функціях, які реалізують основну логіку програми. Проте зі збільшенням кількості сутностей та операцій такий підхід призводить до змішування коду, що відповідає за обробку даних, із кодом взаємодії зі СКБД. Для організації доступу до даних доцільно використовувати патерн Repository (Репозиторій).

Repository – це архітектурний патерн, який передбачає створення окремого компонента для отримання та збереження об'єктів предметної області.

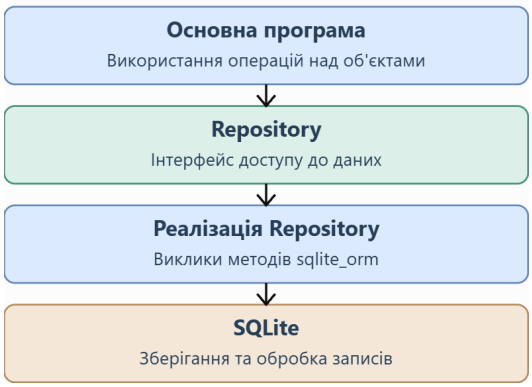
Основна ідея полягає в тому, що інші компоненти програми звертаються до репозиторію через визначений набір операцій, не використовуючи безпосередньо механізми роботи з базою даних.

Наприклад, у системі обліку обладнання можуть виконуватися такі операції:

- додавання нового пристрою;
- отримання інформації про пристрій за ідентифікатором;
- отримання переліку обладнання;
- оновлення характеристик пристрою;
- видалення запису про обладнання.

Ці операції можна об'єднати в окремому компоненті EquipmentRepository.

Інші частини програми використовуватимуть методи цього компонента, не формуючи SQL-запити та не звертаючись безпосередньо до sqlite_orm.



Важливо розрізняти призначення ORM та Repository. ORM забезпечує перетворення між об'єктами C++ і таблицями реляційної бази даних. Repository визначає спосіб організації операцій доступу до цих об'єктів у структурі програми. Отже, Repository може використовувати ORM для виконання операцій, але не є частиною самої технології об'єктно-реляційного відображення.

Визначення інтерфейсу репозиторію

У C++ інтерфейс репозиторію можна реалізувати за допомогою абстрактного класу, який містить чисто віртуальні методи.

Розглянемо інтерфейс для роботи з об'єктами Equipment:

```
class IEquipmentRepository
{
public:
    virtual ~IEquipmentRepository() = default;

    virtual int add(
        const Equipment& equipment) = 0;

    virtual std::optional<Equipment> findById(
        int id) const = 0;

    virtual std::vector<Equipment> findAll()
        const = 0;

    virtual void update(
        const Equipment& equipment) = 0;

    virtual void remove(int id) = 0;
};
```

Інтерфейс визначає набір операцій, доступних для роботи з обладнанням.

Метод add() додає новий об'єкт і повертає ідентифікатор створеного запису.

Метод findById() призначений для пошуку об'єкта за первинним ключем. Оскільки запис може бути відсутнім, результат представлено через std::optional<Equipment>.

Метод findAll() повертає колекцію об'єктів Equipment.

Методи update() та remove() забезпечують зміну й видалення записів.

Усі методи оголошені чисто віртуальними за допомогою специфікатора = 0. Це означає, що абстрактний клас визначає лише їхні сигнатури, а реалізація повинна бути надана в похідному класі.

Віртуальний деструктор забезпечує коректне знищення об'єктів похідних класів через вказівник на базовий клас.

Звернемо увагу, що інтерфейс не містить типів SQLite або sqlite_orm. Усі операції описані через стандартні типи C++ та модель предметної області. Це дозволяє використовувати один інтерфейс із різними реалізаціями механізму зберігання даних.

Реалізація Repository з використанням ORM

Для виконання операцій із базою даних необхідно створити клас, який реалізує визначений інтерфейс.

У нашому прикладі таким класом може бути SQLiteEquipmentRepository.

```
class SQLiteEquipmentRepository
    : public IEquipmentRepository
{
public:
    int add(
        const Equipment& equipment) override;
    std::optional<Equipment> findById(
        int id) const override;
    std::vector<Equipment> findAll()
        const override;
    void update(
        const Equipment& equipment) override;
    void remove(int id) override;
};
```

Ключове слово override вказує, що відповідний метод перевизначає віртуальний метод базового класу. Реалізація методів використовує можливості sqlite_orm.

Наприклад, логіка методу findById() може бути представлена так:

```
std::optional<Equipment>
SQLiteEquipmentRepository::findById(
    int id) const
{
    return storage.get_optional<Equipment>(id);
}
```

У цьому фрагменті передбачається, що storage є доступним членом класу або іншим належним чином організованим об'єктом сховища. Його повне оголошення буде наведено в практичному прикладі.

Аналогічно реалізуються інші операції.

Метод Repository	Операція sqlite_orm
add()	insert()
findById()	get_optional()
findAll()	get_all()
update()	update()
remove()	remove()

Таким чином, репозиторій виступає окремим компонентом, який інкапсулює звернення до ORM. У складніших застосунках його методи можуть виконувати додаткові дії: формувати спеціалізовані запити, обробляти помилки доступу до даних або координувати декілька операцій зі сховищем.

Водночас Repository не повинен перетворюватися на компонент, який реалізує всі правила предметної області. Наприклад, перевірка того, чи дозволено переводити

обладнання зі стану repair у стан active, є правилом роботи системи, а не безпосередньою операцією зберігання даних.

Після створення конкретної реалізації репозиторію основна програма може працювати з ним через інтерфейс базового класу.

Наприклад:

```
void printEquipment(
    const IEquipmentRepository& repository)
{
    auto items = repository.findAll();
    for (const auto& item : items)
    {
        std::cout << item.id << " "
                  << item.name << " "
                  << item.status << '\n';
    }
}
```

Функція printEquipment() отримує посилання на IEquipmentRepository. Вона не знає, яка саме реалізація використовується для отримання даних. Для функції важливо лише те, що переданий об'єкт підтримує метод findAll() і повертає колекцію Equipment. У результаті програмний код, який використовує дані, відокремлюється від деталей їхнього зберігання.

Використання Repository забезпечує низку практичних переваг.

1. Відокремлення доступу до даних. Операції взаємодії з базою даних зосереджуються в окремому компоненті, що спрощує структуру основної програми.

2. Зменшення дублювання коду. Однакові операції отримання або збереження об'єктів не потрібно повторювати в різних частинах застосунку.

3. Спрощення модифікації. Зміни в реалізації доступу до даних можуть бути локалізовані в репозиторії без необхідності переписувати весь програмний код.

4. Можливість використання різних реалізацій. Один інтерфейс може використовуватися з різними способами зберігання даних.

5. Спрощення тестування. Для перевірки роботи інших компонентів програми можна використовувати спеціальну тестову реалізацію репозиторію, яка не потребує реальної бази даних.

Однак створення Repository не є обов'язковим для кожного застосунку. У невеликих програмах із простою логікою додатковий рівень абстракції може збільшувати кількість класів і файлів без суттєвої практичної користі. Тому доцільність використання цього патерну визначається складністю системи, кількістю операцій із даними та вимогами до супроводу й тестування.

Repository та подальше тестування застосунку

Під час розроблення програмної системи важливо передбачати можливість перевірки її компонентів. Якщо всі операції доступу до бази даних зосереджені безпосередньо в основній програмі, тестування може потребувати створення та налаштування реальної бази даних навіть для перевірки простих алгоритмів. Використання Repository дозволяє створити альтернативну реалізацію інтерфейсу, яка зберігатиме об'єкти в оперативній пам'яті.

Наприклад, замість SQLite можна використати контейнер

```
std::unordered_map<int, Equipment> equipment;
```

Така реалізація може бути корисною для модульного тестування, оскільки дозволяє задавати початкові дані без звернення до СКБД.

Водночас тестова реалізація не перевіряє правильність SQL-запитів, зовнішніх ключів або транзакцій. Для цього необхідні окремі інтеграційні тести з використанням реальної бази даних.

1.4. Транзакції та забезпечення цілісності даних

Під час роботи з реляційними базами даних окремі операції часто є логічно пов'язаними. Наприклад, реєстрація нового обладнання може передбачати одночасне створення запису про пристрій та внесення інформації про його початкове технічне обслуговування.

Якщо одна з операцій завершиться успішно, а інша – помилкою, база даних може перейти в стан, який не відповідає правилам предметної області. Для забезпечення узгодженого виконання таких операцій використовуються транзакції (transactions).

Транзакція – це послідовність однієї або декількох операцій із базою даних, які розглядаються як єдина логічна одиниця роботи.

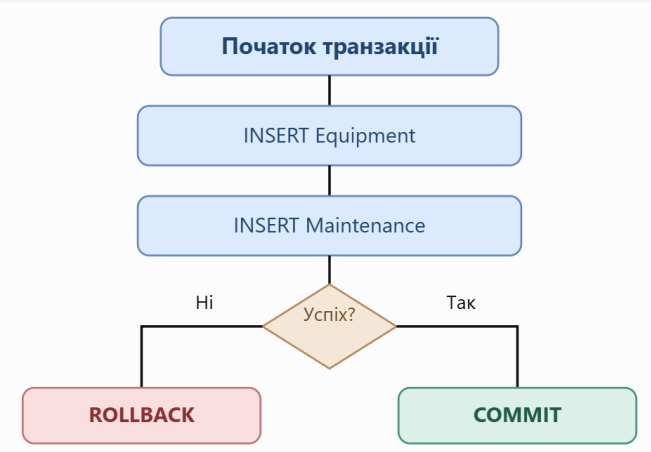
Основна властивість транзакції полягає в тому, що її зміни або фіксуються разом, або скасовуються.

Розглянемо приклад реєстрації обладнання. Програма повинна виконати дві операції:

1. Додати новий пристрій до таблиці Equipment.
2. Додати початковий запис обслуговування до таблиці Maintenance.

Якщо обидві операції виконуються успішно, зміни зберігаються в базі даних.

Якщо друга операція завершується помилкою, необхідно скасувати також і першу, щоб не залишити частково виконану реєстрацію.



Для керування транзакціями в SQL використовуються такі операції:

- BEGIN – початок транзакції;
- COMMIT – фіксація виконаних змін;
- ROLLBACK – скасування змін поточної транзакції.

Після успішного виконання COMMIT зміни стають зафіксованими.

Якщо виконується ROLLBACK, база даних повертається до стану, який існував до початку транзакції, з урахуванням правил транзакційної обробки СКБД.

Надійність транзакційної обробки традиційно характеризується властивостями ACID.

➢ Atomicity (Атомарність). Транзакція виконується як неподільна логічна одиниця. Якщо її неможливо завершити успішно, зміни скасовуються.

➢ Consistency (Узгодженість). Успішно завершена транзакція повинна переводити базу даних з одного допустимого стану в інший. Для цього необхідно дотримуватися визначених обмежень цілісності та правил предметної області.

➢ Isolation (Ізольованість). Паралельне виконання транзакцій не повинно призводити до некоректного взаємного впливу їхніх проміжних станів. Конкретний механізм ізоляції залежить від СКБД.

➢ Durability (Довговічність). Після успішної фіксації транзакції зміни повинні зберігатися відповідно до гарантій СКБД навіть у разі подальшого аварійного завершення програми.

Виконання транзакцій у sqlite_orm

Бібліотека sqlite_orm надає метод transaction(), який дозволяє виконувати декілька операцій із базою даних у межах однієї транзакції. У спрощеному вигляді використання цього методу має такий вигляд:

```
bool committed = storage.transaction([&]()
{
    // Операції з базою даних

    return true;
});
```

Метод transaction() приймає функцію, яка виконує необхідні дії над базою даних.

Повернене значення цієї функції визначає результат транзакції:

- true – зміни необхідно зафіксувати;
- false – зміни необхідно скасувати.

Наприклад:

```
bool committed = storage.transaction([&]()
{
    storage.insert(Equipment{
        0, "Sensor A1", "Sensor", "active"
    });
    return true;
});
```

У наведеному випадку після успішного додавання об'єкта транзакція фіксується. Якщо замість true повернути false, виконані в межах транзакції зміни будуть скасовані. У разі виникнення виключення під час виконання функції transaction() бібліотека виконує відкат транзакції та передає виключення далі для обробки програмою.

Це дозволяє використовувати звичайні механізми обробки виключень C++.

Обробка помилок під час транзакцій

Помилки виконання операцій із базою даних можуть виникати з різних причин:

- порушення обмежень первинного або зовнішнього ключа;
- порушення обмеження унікальності;
- спроба збереження недопустимого значення;
- блокування бази даних;
- помилки доступу до файлу бази даних.

Для обробки таких ситуацій використовуються конструкції try і catch.

Наприклад:

```
try
{
    bool committed = storage.transaction([&]()
```

```
        {
            // Послідовність операцій із базою даних

            return true;
        });

        if (committed)
        {
            std::cout << "Transaction committed\n";
        }
    }
    catch (const std::exception& ex)
    {
        std::cerr << "Transaction failed: "
                  << ex.what() << '\n';
    }
```

Якщо всередині транзакції виникне виключення, її зміни будуть скасовані, а керування перейде до відповідного обробника.

Слід розрізняти два випадки завершення транзакції. У першому випадку програма самостійно визначає, що зміни не потрібно фіксувати, і повертає false. У другому випадку операція завершується помилкою та генерує виключення. Обидва випадки можуть призвести до відкату транзакції, але мають різний зміст для прикладної програми.

Забезпечення посилальної цілісності

Під час роботи з пов'язаними таблицями особливе значення має забезпечення посилальної цілісності. Посилальна цілісність означає, що значення зовнішнього ключа повинно відповідати допустимому запису пов'язаної таблиці, якщо інше не передбачено схемою. Наприклад, запис обслуговування не повинен посилатися на обладнання, якого немає в таблиці Equipment. Для цього в SQLite використовується обмеження FOREIGN KEY.

Однак існує важлива особливість: у SQLite перевірку зовнішніх ключів необхідно ввімкнути для кожного з'єднання, у якому вона потрібна. На рівні SQL це виконується командою:

```
PRAGMA foreign_keys = ON;
```

У sqlite_orm для керування відповідним параметром передбачений інтерфейс pragma.

Якщо перевірка зовнішніх ключів вимкнена, спроба додати запис із неіснуючим значенням зовнішнього ключа може не викликати помилки. Тому під час демонстрації відкату транзакції важливо переконатися, що порушення обмеження справді призводить до виключення.

Перевірка атомарності транзакції

Одним із завдань лабораторної роботи є експериментальна перевірка атомарності транзакції. Для цього необхідно створити ситуацію, у якій перша операція виконується успішно, а друга навмисно завершується помилкою.

Наприклад:

1. Розпочати транзакцію.
2. Додати новий запис до таблиці Equipment.
3. Спробувати додати запис до таблиці Maintenance із недопустимим значенням зовнішнього ключа.
4. Обробити виключення.

5. Перевірити, чи залишився запис про обладнання в базі даних.

Якщо транзакція реалізована правильно, після помилки друга операція не буде виконана, а зміни першої операції будуть скасовані. Важливо, що сам факт виникнення виключення ще не є достатнім доказом правильності відкату. Необхідно виконати додатковий запит до бази даних і переконатися, що запис, створений першою операцією, відсутній. Для наочності доцільно порівняти стан бази даних до початку транзакції та після її завершення.

Ситуація	**Результат**
Обидві операції виконані успішно	Зміни обох таблиць зафіксовано
Друга операція завершилася помилкою	Зміни обох операцій скасовано
Функція транзакції повернула false	Зміни транзакції скасовано

Таким чином, перевірка атомарності передбачає не лише програмну реалізацію транзакції, а й аналіз фактичного стану бази даних.

Поняття атомарності транзакцій має певну аналогію з гарантіями безпеки виключень у C++. Зокрема, сильна гарантія безпеки виключень (strong exception guarantee) передбачає, що в разі невдалого виконання операції спостережуваний стан об'єкта залишається незмінним. Атомарність транзакції забезпечує подібну властивість стосовно змін бази даних: якщо транзакція не була зафіксована, її зміни скасовуються.

Однак ці поняття не є тотожними. Сильна гарантія безпеки виключень стосується стану програмних об'єктів, тоді як транзакція забезпечує атомарність операцій над базою даних. Наприклад, відкат транзакції SQLite не скасовує автоматично зміни звичайних змінних C++, виконані всередині транзакційної функції. Так само транзакція не може автоматично скасувати довільні зовнішні дії, наприклад запис до іншого файлу або надсилання повідомлення.

Отже, під час розроблення застосунку необхідно розрізняти стан бази даних і стан об'єктів програми.

2. Завдання до лабораторної роботи

2.1. Загальне завдання

Розробити консольний застосунок для зберігання та обробки інформації відповідно до індивідуального варіанта предметної області.

Для зберігання даних використати реляційну СУБД SQLite, а для організації доступу до них – бібліотеку sqlite_orm.

Застосунок повинен забезпечувати створення бази даних, виконання основних операцій над об'єктами предметної області, отримання даних за визначеними умовами та виконання взаємопов'язаних операцій у межах транзакції.

Доступ до основної сутності предметної області необхідно організувати з використанням патерну Repository.

Окремо необхідно реалізувати однакові операції за допомогою SQLite C API та sqlite_orm і порівняти ці підходи.

2.2. Вимоги до моделі даних

Відповідно до індивідуального варіанта необхідно визначити дві взаємопов'язані сутності предметної області та представити їх у вигляді структур або класів C++.

Між сутностями повинен існувати зв'язок типу «один-до-багатьох» (1:N).

Перша сутність є основною (батьківською), друга – залежною (дочірньою).

Модель даних повинна відповідати таким вимогам:

1. Основна сутність повинна містити первинний ключ і щонайменше три змістовні поля, які характеризують об'єкт предметної області.
2. Залежна сутність повинна містити первинний ключ, зовнішній ключ для зв'язку з основною сутністю та щонайменше три змістовні поля.
3. Серед полів залежної сутності необхідно передбачити хоча б одне числове поле, придатне для виконання агрегатних обчислень.
4. Для кожної сутності необхідно визначити відповідну таблицю SQLite.
5. Зв'язок між таблицями повинен бути реалізований через зовнішній ключ.
6. Відображення полів C++ на стовпці таблиць необхідно визначити засобами sqlite_orm.
7. Створення та початкове узгодження структури бази даних виконати за допомогою sync_schema().

Для перевірки роботи застосунку необхідно підготувати тестовий набір даних, який містить щонайменше 5 записів основної сутності та 15 записів залежної сутності.

Записи повинні мати різні значення полів, щоб забезпечити можливість перевірки фільтрації, сортування та агрегатних обчислень.

2.3. Порядок виконання роботи

Завдання 1. Створення та налаштування проєкту

Створити проєкт консольного застосунку C++20 із використанням CMake.

Для керування зовнішніми залежностями використати пакетний менеджер vcpkg у режимі manifest.

Необхідно:

1. Створити файл vcpkg.json і визначити необхідні залежності.
2. Налаштувати CMakeLists.txt для складання застосунку.
3. Підключити бібліотеки SQLite та sqlite_orm.
4. Перевірити правильність конфігурації та успішність складання проєкту.

Завдання 2. Порівняння SQLite C API та ORM

Реалізувати двома способами операції додавання одного запису основної сутності та отримання запису за його ідентифікатором:

- безпосередньо через SQLite C API;
- за допомогою бібліотеки sqlite_orm.

Для реалізації через SQLite C API використати підготовлені SQL-запити (prepared statements) та прив'язування параметрів.

Для реалізації через ORM використати відповідні методи бібліотеки.

Порівняти обидва підходи за такими характеристиками: кількість програмного коду, необхідність ручного перетворення даних, керування ресурсами, обробка помилок і зручність супроводу.

Результати порівняння навести у звіті.

Завдання 3. Створення моделі даних та ORM-відображення

Реалізувати структури або класи для двох сутностей індивідуального варіанта.

Визначити відображення цих типів на таблиці SQLite за допомогою make_storage(), make_table() та make_column().

Необхідно:

1. Визначити первинні ключі таблиць.
2. Створити зв'язок між таблицями через зовнішній ключ.
3. Забезпечити перевірку обмежень зовнішніх ключів.
4. Виконати синхронізацію схеми бази даних.
5. Додати початковий набір даних.
6. Перевірити правильність створення таблиць і зв'язків між ними.

Завдання 4. Організація доступу до даних за допомогою Repository

Для основної сутності предметної області реалізувати патерн Repository.

Необхідно:

1. Створити абстрактний клас, який визначає інтерфейс доступу до даних.
2. Передбачити в інтерфейсі методи додавання, отримання за ідентифікатором, отримання всіх записів, оновлення та видалення.
3. Створити похідний клас, який реалізує ці операції за допомогою sqlite_orm.
4. Для отримання запису за ідентифікатором використати get_optional().
5. Продемонструвати роботу репозиторію через інтерфейс базового класу.

Операції доступу до залежної сутності реалізувати в окремому компоненті роботи з даними. Створення для нього додаткового абстрактного інтерфейсу не вимагається.

Завдання 5. Виконання запитів до бази даних

Реалізувати засобами sqlite_orm такі запити:

1. Фільтрація: отримання записів за значенням одного або декількох полів.
2. Сортування та обмеження: отримання впорядкованої вибірки з обмеженою кількістю записів.
3. Агрегація: обчислення одного агрегатного показника за допомогою COUNT, SUM або AVG.
4. Об'єднання таблиць: отримання даних із двох пов'язаних таблиць за допомогою JOIN.

Для запиту з об'єднанням таблиць результат повинен містити поля обох сутностей.

Кожен запит необхідно продемонструвати на підготовленому наборі даних.

Завдання 6. Реалізація транзакції

Реалізувати операцію, яка передбачає послідовну зміну даних у двох пов'язаних таблицях.

Обидві зміни повинні виконуватися в межах однієї транзакції.

Необхідно:

1. Реалізувати успішний сценарій виконання транзакції.
2. Створити сценарій, у якому перша операція виконується успішно, а друга навмисно завершується помилкою через порушення обмеження бази даних.
3. Забезпечити обробку виключення.
4. Перевірити, що в разі помилки всі зміни транзакції скасовано.
5. Порівняти стан бази даних до початку та після завершення кожного сценарію.

Результати перевірки атомарності транзакції навести у звіті.

2.4. Загальні вимоги до програмної реалізації

Програма повинна відповідати таким вимогам:

1. Використовувати стандарт C++20.
2. Застосовувати CMake для складання проєкту та vcpkg для керування залежностями.

3. Містити окремі компоненти для моделей даних, налаштування ORM-відображення та реалізації Repository.

4. Не розміщувати весь код програми в одному файлі main.cpp.

5. Забезпечувати обробку помилок доступу до бази даних.

6. Не використовувати ручне формування SQL-запитів для реалізації основних операцій ORM. Безпосередній SQLite C API застосовувати лише в порівняльній частині роботи.

7. Забезпечувати можливість повторного запуску демонстраційних сценаріїв без неконтрольованого накопичення тестових записів.

8. Виводити результати виконання основних операцій у консоль.

Розроблення інтерактивного меню не є обов'язковим. Допускається реалізація послідовних демонстраційних сценаріїв, які перевіряють усі передбачені завданням операції.

Структура проєкту повинна дозволяти надалі додати модульні тести для компонентів доступу до даних.

Варіанти завдання

№	Предметна область	Основна сутність (1)	Залежна сутність (N)
1	Система обробки журналів подій	Джерело подій	Запис журналу
2	Система обліку роботи кол-центру	Оператор	Дзвінок
3	Конвеєр обробки зображень	Пакет зображень	Завдання обробки
4	Система обробки замовлень інтернет-магазину	Клієнт	Замовлення
5	Система моніторингу температури	Датчик	Вимірювання
6	Система керування чергою друку	Користувач	Документ друку
7	Система обміну повідомленнями	Користувач	Повідомлення
8	Система обліку банківських операцій	Рахунок	Банківська операція
9	Система моніторингу серверів	Сервер	Запис метрики
10	Система обліку замовлень ресторану	Стіл	Замовлення
11	Система керування завантаженням файлів	Користувач	Запит завантаження
12	Система відеоспостереження	Камера	Зареєстрована подія
13	Система обліку показань лічильників	Будинок	Показання лічильника
14	Система керування чергою в поліклініці	Лікар	Талон прийому
15	Система обліку біржових заявок	Трейдер	Заявка
16	Система обробки сповіщень	Категорія сповіщень	Сповіщення
17	Система моніторингу виробничої лінії	Тип виробу	Виготовлена деталь
18	Система моніторингу погодних станцій	Погодна станція	Спостереження
19	Система обліку виконання CI/CD	Програмний проєкт	Збірка
20	Система бронювання квитків	Рейс	Бронювання
21	Система моніторингу GPS-трекерів	Об'єкт спостереження	Точка треку
22	Система обліку результатів голосування	Кандидат	Голос
23	Система обліку роботи кас супермаркету	Каса	Чек
24	Система обліку поїздок таксі	Водій	Поїздка
25	Система обліку складських посилок	Напрямок доставки	Посилка
26	Система моніторингу якості повітря	Датчик	Показник якості повітря

27	Система обробки звернень до служби підтримки	Клієнт	Тікет
28	Система обліку викликів ліфтів	Ліфт	Виклик
29	Система обліку результатів тестування	Студент	Результат тестування
30	Система моніторингу безпілотних апаратів	Дрон	Вимірювання заряду

Вимоги до виконання індивідуального варіанта

Для визначеної предметної області студент повинен:

1. Розробити модель даних. Визначити поля обох сутностей, їхні типи та призначення. Передбачити первинні ключі й зовнішній ключ залежної сутності.

2. Реалізувати ORM-відображення. Створити відповідні структури або класи C++, описати таблиці SQLite та встановити зв'язок між ними засобами sqlite_orm.

3. Реалізувати Repository. Створити інтерфейс і конкретну реалізацію операцій доступу до основної сутності.

4. Визначити запити відповідно до предметної області. Самостійно обрати умови фільтрації, поле сортування, агрегатний показник і склад результату об'єднання таблиць.

5. Розробити транзакційну операцію. Визначити змістовну для предметної області послідовність із двох взаємопов'язаних операцій та забезпечити їх атомарне виконання.

6. Продемонструвати роботу програми. Перевірити основні операції, запити, успішне виконання транзакції та її відкат у разі помилки.

Особливості окремих варіантів

Для забезпечення однозначного трактування предметної області необхідно врахувати такі уточнення.

Варіант 8. Банківська операція є записом про зміну стану рахунку. Її не слід ототожнювати з транзакцією бази даних, яка використовується для забезпечення атомарності операцій.

Варіант 10. Один стіл ресторану може мати декілька замовлень, створених у різний час. Модель не повинна обмежуватися лише поточним замовленням столу.

Варіант 22. Використовується спрощена навчальна модель обліку голосів за кандидатів. Реалізація механізмів ідентифікації виборців, таємниці голосування та захисту реальних виборчих процесів не передбачається.

Варіант 28. Кожен запис виклику належить конкретному ліфту. Для виклику можна передбачити поверх, час, напрямок руху та інші характеристики.

Вимоги до змісту транзакційної операції

Транзакція повинна моделювати логічно завершену дію, характерну для обраної предметної області.

Наприклад, для системи моніторингу серверів це може бути реєстрація нового сервера разом із початковим записом метрики; для системи погодних станцій – додавання станції та першого спостереження; для системи обліку результатів тестування – реєстрація студента та його початкового результату.

Не допускається реалізація транзакції як довільної послідовності не пов'язаних між собою операцій, які не мають спільного змісту в межах предметної області.

Для демонстрації відкату необхідно навмисно створити помилку виконання другої операції та перевірити, що зміни першої операції не були збережені.

Вимоги до програмної реалізації

Застосунок повинен відповідати таким вимогам:

- використовувати стандарт C++20;
- застосовувати CMake для організації збирання проєкту;
- використовувати vcpkg для керування зовнішніми залежностями;
- забезпечувати логічний поділ коду на заголовкові файли та файли реалізації;
- відокремлювати моделі даних, налаштування ORM, реалізацію Repository та демонстраційні сценарії;
- передбачати обробку помилок роботи з базою даних;
- виводити результати виконання операцій у зрозумілому вигляді.

Програмний код повинен бути структурованим, а назви класів, методів і змінних — відповідати їхньому призначенню.

Вимоги до звіту

Звіт повинен містити такі складові:

1. Тема та мета лабораторної роботи. Номер і назва індивідуального варіанта.

2. Опис предметної області. Коротка характеристика предметної області, призначення розробленого застосунку та основні операції, які він виконує.

3. Структура бази даних. Опис таблиць, їхніх полів, первинних і зовнішніх ключів. Схема зв'язку між таблицями із зазначенням типу відношення.

4. Організація програмного проєкту. Короткий опис структури проєкту, використаних бібліотек і засобів збирання. Пояснення призначення основних класів і файлів.

5. Реалізація доступу до даних. Опис використання SQLite C API та sqlite_orm. Навести приклади створення таблиць, відображення об'єктів на таблиці та виконання операцій із даними.

6. Реалізація Repository та CRUD. Опис інтерфейсу репозиторію, його реалізації та результатів виконання операцій додавання, отримання, оновлення й видалення записів.

7. Результати виконання запитів. Навести приклади та результати фільтрації, сортування, обмеження кількості записів, агрегатних обчислень, групування та об'єднання таблиць. Пояснити призначення реалізованих запитів.

8. Робота з транзакціями. Продемонструвати результати успішного виконання транзакції (COMMIT), скасування змін (ROLLBACK) та виконання пов'язаних операцій без явної транзакції. Порівняти отримані результати.

9. Порівняння SQLite C API та ORM. Проаналізувати відмінності між підходами з погляду складності програмного коду, зручності роботи з даними, керування SQL-запитами та можливостей супроводу застосунку.

10. Висновки. Узагальнити результати виконання лабораторної роботи, зазначити особливості використання ORM, Repository та транзакцій.

11. Посилання на GitHub-репозиторій. Репозиторій повинен містити вихідний код застосунку, файли конфігурації CMake та vcpkg, а також інструкцію зі збирання та запуску проєкту.

Повний програмний код у звіті наводити не потрібно. Достатньо представити найбільш показові фрагменти реалізації, пояснення до них та результати виконання. Повний код має бути доступним у GitHub-репозиторії.

Контрольні питання

1. Що таке об'єктно-реляційне відображення (ORM)? Які основні завдання вирішує ORM під час розробки застосунків, що працюють із реляційними базами даних?

2. У чому полягають відмінності між безпосереднім використанням SQLite C API та роботою з базою даних через sqlite_orm? Які переваги й обмеження має кожен підхід?

3. Яке призначення функцій make_storage(), make_table(), make_column() та методу sync_schema() у бібліотеці sqlite_orm?

4. Як реалізується зв'язок типу «один до багатьох» між таблицями? Яку роль відіграють первинний і зовнішній ключі та як забезпечується перевірка посилальної цілісності в SQLite?

5. У чому полягає призначення патерна Repository? Чому доцільно відокремлювати логіку доступу до даних від основної логіки застосунку?

6. Яким чином реалізуються CRUD-операції засобами sqlite_orm? Як необхідно обробляти ситуації, коли запис для оновлення, отримання або видалення відсутній?

7. Як реалізуються фільтрація, сортування та обмеження кількості результатів у sqlite_orm? Які SQL-конструкції відповідають цим операціям?

8. Для чого використовуються агрегатні функції COUNT, SUM, AVG та оператор GROUP BY? Чим агрегатні запити відрізняються від запитів, які повертають окремі записи?

9. Яке призначення оператора INNER JOIN? Як за допомогою sqlite_orm отримати дані з двох пов'язаних таблиць?

10. Що таке транзакція? Поясніть властивості ACID та їхнє значення для забезпечення надійності роботи з базою даних.

11. У чому полягає відмінність між операціями COMMIT і ROLLBACK? Як реалізується керування транзакціями за допомогою storage.transaction()?

12. Чому виконання декількох пов'язаних операцій без явної транзакції може призвести до часткового збереження даних? Наведіть приклад ситуації, у якій використання транзакції є необхідним.